



VELTRAXX'26



TEAM NAME : Chip Smiths

PROBELM STATEMENT NO : 11

Problem Statement Title : Sparse Weight Pruning and Run-Length Encoding Engine for Edge NPU

PS Category - Edge AI / TinyML

Software



Edit with WPS Office





VELTRAXX'26



Problem Statement

- Real-world problem: Edge AI models require large memory and computation, making deployment on resource-constrained devices difficult.
- Why it matters: Reducing model size and computation enables faster, more efficient, and low-memory AI inference on edge devices.
- Current challenges: Neural networks contain many redundant weights, while conventional storage and computation still process these unnecessary values.



Edit with WPS Office





VELTRAXX'26



yOur Solution

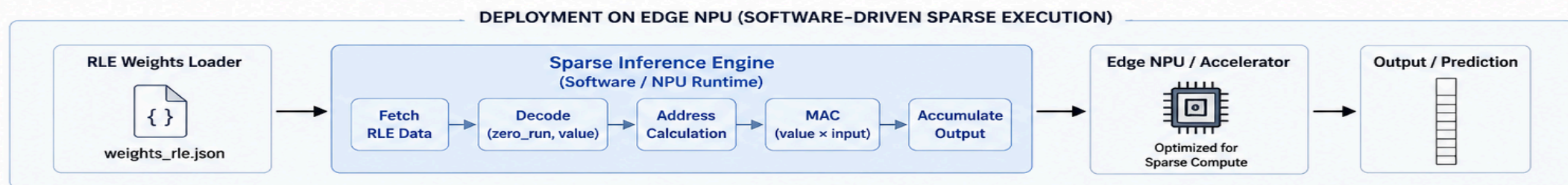
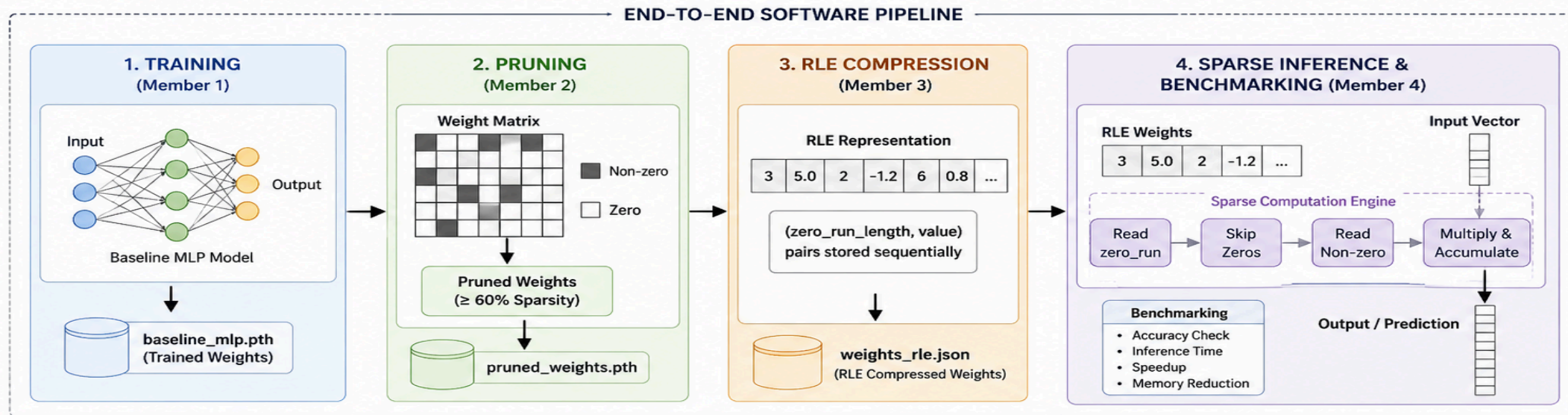
- Project name : Sparse Weight Pruning and Run-Length Encoding Engine for Edge NPU
- One-line solution : “Prune unnecessary neural-network weights and compress the remaining sparse weights using Run-Length Encoding to reduce memory and computation during edge inference.”
- How it solves the problem : “It solves the problem by removing redundant weights, encoding the resulting sparse data efficiently, and reducing the workload required for neural-network inference.”



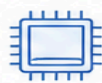
Edit with WPS Office



Sparse Weight Pruning and Run-Length Encoding Engine for Edge NPU



SPARSITY
≥ 60%
weights set to zero



MEMORY REDUCTION
≥ 2×
less memory usage



COMPUTATIONAL SPEEDUP
≥ 1.5×
faster inference



ACCURACY
Close to baseline
model accuracy

Member 1: Training

Member 2: Pruning

Member 3: RLE Compression

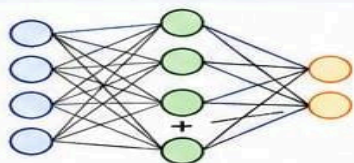
Member 4: Sparse Inference

→ Data Flow

Sparse Weight Pruning and Run-Length Encoding Engine for Edge

OVERALL SYSTEM FLOW

MEMBER 1 TRAIN BASELINE MLP



Goal: Train a baseline MLP on MNIST dataset

1. Load MNIST Dataset (60,000 train / 10,000 test)

2. Preprocess Data
- Normalize to [0,1]
- Flatten 28x28 → 784

3. Define MLP Architecture
Input: 784
- Hidden: 128 (ReLU)
Output: 10 (Softmax)

4. Train Model
- Loss: Cross Entropy
- Optimizer: Adam
- Epochs: 20
- Batch Size: 128

5. Evaluate Model
- Test Accuracy
- Loss

6. Save Model
baseline_mlp.pth

OUTPUT
Trained Baseline Model

baseline_mlp.pth

MEMBER 2 PRUNE WEIGHTS

0.5	0	-1.2	0
0	2.1	0	0
-0.3	0	0.8	0
0	0	0	1.5

Dense Weights

→

0.5	0	-1.2	0
0	2.1	0	0
0	0	0.8	0
0	0	0	1.5

Pruned Weights

Goal: Achieve ≥60% sparsity using magnitude-based pruning

1. Load Baseline Model
baseline_mlp.pth

2. Extract All Weight Matrices
(from both layers)

3. Compute Global Threshold
- Flatten all weights
- Sort by |value|
- Find threshold for 60%

4. Prune Weights
If |weight| < threshold → set to 0

5. Calculate Sparsity
 $\text{Sparsity} = (\text{Zeros} / \text{Total}) \times 100\%$

Is Sparsity ≥ 60%?

NO → Adjust Threshold (Decrease)

YES → 6. Save Pruned Model

6. Save Pruned Model
pruned_mlp.pth

OUTPUT
Pruned Model (≥60% Sparse)

pruned_mlp.pth

Sparse Weight Pruning and Run-Length Encoding Engine for Edge NPU

OVERALL SYSTEM FLOW

MEMBER 3 RLE COMPRESSION

Example (Flattened Weights)



Goal: Compress pruned weights using Run-Length Encoding (RLE)

1. Load Pruned Model
pruned_mlp.pth

2. For Each Layer Weight Matrix
- Flatten the matrix

3. Apply RLE Encoding
- Count consecutive zeros (zero_run)
- Store (zero_run, value) for each non-zero value

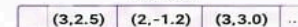
4. Store Metadata
- original_shape
- total_elements
- rle_pairs

5. Save RLE Data
rle_compressed.json

OUTPUT
RLE Compressed Weights
rle_compressed.json

MEMBER 4 SPARSE INFERENCE + BENCHMARKING

RLE Data



Sparse Computation Engine

Output / Prediction

Goal: Perform sparse inference and compare with dense inference

1. Load RLE Data
rle_compressed.json

2. Load Test Data
from MNIST

3. Sparse Inference Engine
- Read RLE pairs
- Skip zero runs
- Multiply only non-zero weights
- Accumulate results

4. Dense Inference (Baseline)
- Use pruned weights in dense form
- Standard matrix multiplication

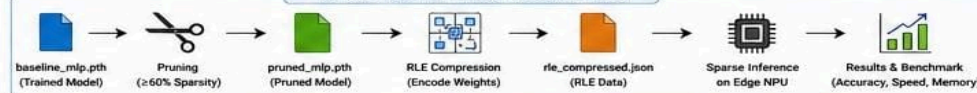
5. Compare Outputs
- Accuracy Check
- Maximum Difference

6. Benchmark Performance
- Measure average time
- Compute Speedup

7. Save Results
inference.json

OUTPUT
Inference Results & Benchmark
inference.json

INTEGRATION PIPELINE (End-to-End Flow)



FINAL BENCHMARK & TARGETS

METRICS

- ✓ 1. Sparsity (%)
- ✓ 2. Memory Reduction (x)
- ✓ 3. Computational Speedup (x)
- ✓ 4. Accuracy (Test Set)

TARGETS (PS11)

Sparsity : ≥ 60 %
Memory Reduction : ≥ 2 ×
Computational Speedup : ≥ 1.5 ×
Accuracy : Should be close to baseline

EXPECTED OUTCOME

- ✓ High sparsity with minimal accuracy loss
- ✓ Reduced memory footprint
- ✓ Faster inference on Edge NPU
- ✓ Efficient deployment for real-time applications

DEPLOYMENT ON EDGE NPU

Efficient • Fast • Lightweight
Power Optimized

Purpose of Blocks

- Input Block – Loads and prepares the neural-network weights and input data.
 - Pruning Block – Removes less important weights to create a sparse model.
 - RLE Compression Block – Compresses consecutive zero weights to reduce storage.
 - Sparse Inference Block – Processes only non-zero weights to reduce computation.
 - Output Block – Produces the prediction and performance results.
-
- IDEA BEHIND CHOOSING EACH BLOCK IN ARCHITECTURAL DIAGRAM AND WHAT ARE THE TRADE OFF OF YOUR ARCHITECTURAL CHOICE :
 - The blocks are arranged to progressively reduce model size and computation: pruning creates sparsity, RLE reduces storage, and sparse inference avoids unnecessary operations. The main trade-off is that higher compression and sparsity can reduce memory and computation but may slightly affect accuracy and add encoding/decoding complexity.

SparQ-NPU: Execution Results

End-to-End Pipeline Execution Outputs

1 MEMBER 1: BASELINE TRAINING

```
> py member1/train.py
Loading dataset...
Training MLP...
Epoch 5/30 | Loss: 2.1944
Epoch 10/30 | Loss: 2.0418
Epoch 15/30 | Loss: 1.8551
Epoch 20/30 | Loss: 1.6302
Epoch 25/30 | Loss: 1.3772
Epoch 30/30 | Loss: 1.1175

===== BASELINE MLP RESULTS =====
Accuracy      : 82.22%
Parameters    : 17226
Memory        : 68904 bytes
Inference time : 0.000166 seconds

Model saved to:
../models/baseline_mlp.pth

Results saved to:
../results/baseline.json
```



BASELINE SUMMARY

- Accuracy: 82.22%
- Parameters: 17226
- Memory: 68,904 bytes
- Inference Time: 0.000166s

2 MEMBER 2: PRUNING

```
> py member2/prune.py
Loading baseline model...
Baseline model loaded successfully.
Applying 60% global pruning...

===== PRUNING RESULTS =====
Original parameters : 17226
Total weights       : 17024
Zero weights        : 10214
Sparsity            : 60.00%
Remaining weights   : 40.00%

Pruned model saved to:
../models/pruned_mlp.pth

Pruning results saved to:
../results/pruning.json
```



PRUNING SUMMARY (60%)

- Sparsity Achieved : 60.00%
- Zero Weights : 10,214
- Remaining Weights: 40.00%
- Model Size Reduced

PIPELINE OVERVIEW (1/2)

1

Baseline Training
Accuracy: 82.22%



2

Pruning (60%)
Sparsity: 60.00%



3

RLE Compression



4

Sparse Inference

3 MEMBER 3: RLE COMPRESSION

```
> py member3/rle.py
Creating sample sparse weight matrix...
===== RLE ENCODING =====
Encoded representation:
Zero run: 0 | Value: 5.0
Zero run: 3 | Value: 2.0
Zero run: 2 | Value: 7.0
Zero run: 2 | Value: 3.0
Zero run: 4 | Value: 4.0
Zero run: 0 | Value: 6.0
Zero run: 4 | Value: 8.0
===== COMPRESSION RESULTS =====
Total elements      : 24
Zero elements       : 17
Non-zero elements   : 7
Sparsity            : 70.83%
Dense memory        : 96 bytes
RLE memory          : 56 bytes
Compression ratio    : 1.71x
Memory reduction    : 41.67%
=====
Results saved to:
../results/compression.json
```

COMPRESSION SUMMARY (RLE)



- Sparsity: 70.83%
- Compression Ratio: 1.71x
- Memory Reduction: 41.67%
- Efficient Storage



4 MEMBER 4: SPARSE INFERENCE

```
> py member4/sparse_inference.py
Member 4 environment is ready
PyTorch version: 2.13.0+cpu

Matrix Shape:      torch.Size([4, 5])
Total elements:    20
Non-zero values:    6
Sparsity:           70.00%

SUCCESS: Decoding matches
the original matrix!

Input Vector:
tensor([1., 2., 3., 4., 5.])

Dense Inference Output:
tensor([ 1.5000, -2.0000, 5.1000, 6.0000])

Sparse Inference Output:
tensor([ 1.5000, -2.0000, 5.1000, 6.0000])

SUCCESS: Sparse inference
matches dense inference!

--- RLE-BASED SPARSE INFERENCE ---
RLE Inference Output:
tensor([ 1.5000, -2.0000, 5.1000, 6.0000])
SUCCESS: RLE inference
matches dense inference!
```

--- COMPUTATION COMPARISON ---

Dense Operations:	20
Sparse Operations:	6
Operations Saved:	14
Operation Reduction:	70.00%

--- STORAGE COMPARISON ---

Dense Storage Entries:	20
RLE Storage Entries:	12
Storage Entries Saved:	8
Storage Reduction:	40.00%

SPARQ-NPU FINAL PERFORMANCE SUMMARY

Total Weights:	20
Non-Zero Weights:	6
Zero Weights:	14
Sparsity:	70.00%

RLE Decoding:	PASSED
Sparse Inference:	PASSED

BENCHMARK STATUS: SUCCESS

--- PERFORMANCE BENCHMARK (10000 ITERATIONS) ---

Dense Time : 0.019455 sec	Avg Time Per Inference:
Sparse Time : 0.478234 sec	Dense : 0.000001946 sec
RLE Time : 0.636978 sec	Sparse : 0.000047823 sec
	RLE : 0.000063698 sec

INFERENCE SUMMARY



- Operation Reduction: 70.00%
- Storage Reduction: 40.00%
- All Verifications PASSED
- End-to-End Success

Edit with WPS Office



VELTRAXX'26



RESEARCH AND REFERENCES:

RESEARCH AND REFERENCES

Scikit-learn – MLP (Multi-Layer Perceptron)

Used for baseline MLP training and evaluation.

[MLP Documentation](#)

PyTorch – Model Pruning

Used for removing unnecessary weights and creating sparsity.

[PyTorch Pruning Tutorial](#)

Deep Compression – Han et al.

Research reference for pruning and neural-network compression.

[Deep Compression Paper](#)

SciPy – Sparse Data Structures

Reference for sparse representation and efficient computation.

[SciPy Documentation](#)



Edit with WPS Office



VELTRAXX'26



1.NO SLIDES SHOULD BE REUSED EXCEPT THE OUTPUT AND RESULT SLIDE (SLIDE 6)

NOTE: MAXIMUM OF 10 SLIDES (NOT MORE THAN THAT)

1. TRY TO EXPLAIN THE CONTENT IN PROBLEM STATEMENT SLIDE 2 AND THE SOLUTION SLIDE 3 IN A NEAT BULLET POINTS AND TRY TO KEEP REST OF PRESENTATIONS WITH MINIMAL TEXT

3.KEEP YOUR EXPLANATION PRECISE AND EASY TO UNDERSTAND

4.IDEA SHOULD BE UNIQUE AND NOVEL.

5.YOU CAN ONLY USE PROVIDED TEMPLATE FOR MAKING THE PPT WITHOUT CHANGING THE IDEA DETAILS POINTERS (MENTIONED IN PREVIOUS SLIDES).

6.YOU NEED TO SAVE THE FILE IN PDF AND UPLOAD THE SAME ON GITHUB . WORD DOC OR ANY OTHER FORMAT WILL BE SUPPORTED

NOTE - YOU CAN DELETE THIS SLIDE (IMPORTANT POINTERS) WHEN YOU UPLOAD THE DETAILS OF YOUR IDEA ON GITHUB.



Edit with WPS Office